

Module 1: Introduction to Geometric Learning

Section 1.2 — Invariant and Equivariant Representations

Data Science Master's Programme

Contents

1	Invariance	1
2	Equivariance	2
3	Why Invariance Alone Is Insufficient	3
4	Constructing Invariant and Equivariant Maps	3
4.1	Averaging (Reynolds operator)	3
4.2	Canonicalisation	3
4.3	Equivariant weight sharing	4
5	A Glimpse of Representation Theory	4
6	The Invariance–Equivariance Tradeoff	5

Motivation

In Section 1.1 we saw that data often comes with natural symmetry groups. The next question is: *how should a learned representation or a model output behave when a symmetry transformation is applied to the input?*

There are two fundamental answers:

- The output **does not change** — this is *invariance*.
- The output **transforms in a predictable, structured way** — this is *equivariance*.

Both notions are central to modern geometric deep learning. Understanding the distinction, and knowing when to use each, is one of the key skills of this module.

1 Invariance

Definition 1.1: G -Invariant Function

Let G act on a set X and let Y be any set. A function $f : X \rightarrow Y$ is **G -invariant** if

$$f(g \cdot x) = f(x) \quad \text{for all } g \in G, x \in X.$$

In words: applying a symmetry transformation to the input leaves the output completely unchanged.

Example 1.1: Classification of images

Let X be the space of images and $G = \text{SO}(2)$ be the group of rotations. A cat-vs-dog classifier $f : X \rightarrow \{\text{cat}, \text{dog}\}$ should ideally be $\text{SO}(2)$ -invariant: rotating a cat image should still produce the label “cat.”

Example 1.2: Graph-level property prediction

In computational chemistry, one often wants to predict a scalar property (e.g. solubility) of a molecule represented as a graph. Since the property does not depend on how we label the atoms, the predictor $f : \mathcal{G} \rightarrow \mathbb{R}$ must be S_n -invariant (invariant to permutations of node indices).

Remark 1.1: Invariant functions are constant on orbits

A function $f : X \rightarrow Y$ is G -invariant if and only if it is constant on every G -orbit. Equivalently, f descends to a well-defined function on the *quotient space* X/G of orbits. This is a useful conceptual picture: invariant predictors do not see individual points, only their equivalence classes under symmetry.

2 Equivariance

Invariance alone is often too rigid: we may want to *track* how the output changes as the input is transformed. This is the role of equivariance.

Definition 2.1: G -Equivariant Map

Let G act on X via ρ_X and on Y via ρ_Y . A function $f : X \rightarrow Y$ is **G -equivariant** if

$$f(\rho_X(g) \cdot x) = \rho_Y(g) \cdot f(x) \quad \text{for all } g \in G, x \in X.$$

When the actions are clear from context, we write this as $f(g \cdot x) = g \cdot f(x)$.

Informally: *it does not matter whether you first transform the input and then apply f , or first apply f and then transform the output — you always get the same result.*

Remark 2.1: Invariance as a special case

Invariance is equivariance where the action on Y is trivial: $\rho_Y(g) \cdot y = y$ for all g, y . So invariance $\subset$ equivariance.

Example 2.1: Feature maps in convolutional networks

Let $X = \mathbb{R}^{H \times W}$ be the space of grayscale images and $G = \mathbb{Z}^2$ act by pixel translation. A convolutional layer $f : X \rightarrow X$ with a fixed filter is *translation-equivariant*: translating the input image translates the feature map by the same amount.

$$f(\text{translate}(x, \delta)) = \text{translate}(f(x), \delta).$$

This is precisely why convolutional networks generalise well across spatial positions without needing to see every object at every location.

Example 2.2: Keypoint detection

A keypoint detector $f : X \rightarrow \mathbb{R}^{2 \times K}$ outputs K 2D coordinates on an image. If the image is rotated by R , the keypoints should rotate by the same R . Formally, f should be $\text{SO}(2)$ -equivariant with the standard action on 2D coordinates. Keypoint detectors used in robotics and pose estimation are engineered to have this property.

Example 2.3: Equivariance in segmentation

A semantic segmentation network $f : \mathbb{R}^{H \times W} \rightarrow \{1, \dots, C\}^{H \times W}$ assigns a class label to every pixel. It should be *translation-equivariant*: shifting the input image by δ should shift the segmentation mask by the same δ . Unlike classification (which is invariant), segmentation is the prototypical example of an equivariant task: the output is a geometric object (a mask) that lives in the same spatial domain as the input.

3 Why Invariance Alone Is Insufficient

One might ask: why not just build invariant networks for every task? The answer is that *invariant representations lose structural information* that is often needed in intermediate layers or for downstream tasks.

Consider building a 3D object detector. The final classification output should be rotation-invariant (a chair is a chair regardless of orientation), but the intermediate features representing *where the chair is pointing* must be rotation-equivariant: if you rotate the input scene, the directional features should rotate accordingly so that the full 3D geometry is preserved throughout the network.

Theorem 3.1: Equivariant $\rightarrow$ Invariant via pooling

If $f : X \rightarrow Y$ is G -equivariant and $\text{pool} : Y \rightarrow Z$ is G -invariant (e.g. global average pooling or sum pooling), then $\text{pool} \circ f$ is G -invariant.

Sketch. Composition of an equivariant map with an invariant one is invariant: $(\text{pool} \circ f)(g \cdot x) = \text{pool}(g \cdot f(x)) = \text{pool}(f(x)) = (\text{pool} \circ f)(x)$. $\square$

This is the standard design pattern in geometric deep learning:

Stack equivariant layers $\rightarrow$ apply invariant pooling at the end.

4 Constructing Invariant and Equivariant Maps

4.1 Averaging (Reynolds operator)

Given any function $f : X \rightarrow Y$ (with Y a vector space), define the **symmetrisation**:

$$\bar{f}(x) = \frac{1}{|G|} \sum_{g \in G} f(g \cdot x) \quad (\text{for finite } G).$$

Then $\bar{f}$ is G -invariant. This operator is called the *Reynolds operator* and is the simplest way to build invariance into an arbitrary function.

Example 4.1: Invariant layer from a linear map

Let $G = S_3$ act on $\mathbb{R}^3$ by permuting coordinates. Start with a linear map $f(x) = Wx$. The symmetrised version $\bar{f}(x) = \frac{1}{6} \sum_{\sigma \in S_3} W P_\sigma x$ is permutation-invariant. One can show that after averaging, $\bar{f}$ reduces to a linear combination of the *sum* of the input coordinates — the simplest permutation-invariant statistic.

4.2 Canonicalisation

Another strategy: pick a *canonical representative* for each orbit and always evaluate f on that representative.

$$f_{\text{canon}}(x) = f(\text{canon}(x)), \quad \text{where } \text{canon}(x) \in G \cdot x \text{ is fixed.}$$

This is used, for instance, in *canonical forms* for graphs (e.g. Nauty/Traces algorithms) and in *pose normalisation* for 3D point clouds (aligning via PCA before processing). The downside is that canonicalisation can be discontinuous and expensive to compute.

4.3 Equivariant weight sharing

For neural networks, equivariance can be built in by *constraining the weights* so that the layer commutes with the group action. The classical example is the convolutional layer: the weight-sharing structure is exactly the constraint that forces translation equivariance. More general constructions (e.g. equivariant linear layers for S_n , $\text{SO}(3)$) follow the same principle but require more algebra (representation theory), which we introduce next.

Remark 4.1: Comparing the three strategies

Each strategy has a characteristic trade-off: *Reynolds averaging* is universal but expensive (factor $|G|$ per forward pass), and cannot exploit shared computation across orbit elements. *Canonicalisation* sidesteps that cost by computing f once per input, but suffers from discontinuities at orbit boundaries (the canonical representative jumps when the input crosses a symmetry axis), which hurts gradient-based training. *Weight sharing* is the most powerful: it preserves continuity, enables end-to-end training, and is exactly equivalent (under mild conditions) to Reynolds averaging applied to the layer's parameters rather than to its outputs.

5 A Glimpse of Representation Theory

How do groups act on *vector spaces*?

Definition 5.1: Linear Representation

A **linear representation** of a group G on a vector space V is a group homomorphism $\rho : G \rightarrow \text{GL}(V)$, i.e. a map assigning an invertible linear transformation $\rho(g) \in \text{GL}(V)$ to each $g \in G$, such that

$$\rho(g \cdot h) = \rho(g) \circ \rho(h).$$

Example 5.1: Standard representation of S_n

Map each permutation $\sigma \in S_n$ to its $n \times n$ permutation matrix P_σ : $(P_\sigma)_{ij} = 1$ iff $\sigma(j) = i$. This is the *standard (defining) representation* of S_n on $\mathbb{R}^n$. Equivariant linear maps

$\mathbb{R}^n \rightarrow \mathbb{R}^n$ with respect to this representation are exactly the matrices that commute with all P_σ — one can show these are scalar multiples of the identity plus multiples of the all-ones matrix.

Definition 5.2: Irreducible representation

A representation $\rho : G \rightarrow \text{GL}(V)$ is **irreducible** if V has no proper non-trivial G -invariant subspace, i.e. no subspace $W \subsetneq V$ with $W \neq \{0\}$ satisfying $\rho(g)W \subseteq W$ for all $g \in G$. Irreducible representations (or *irreps*) are the elementary building blocks: every finite-dimensional representation of a finite or compact group decomposes as a direct sum of irreps.

Theorem 5.1: Schur’s Lemma (simplified)

Let $\rho_1 : G \rightarrow \text{GL}(V_1)$ and $\rho_2 : G \rightarrow \text{GL}(V_2)$ be *irreducible* representations (representations with no nontrivial invariant subspace). Any equivariant linear map $f : V_1 \rightarrow V_2$ is either

- (i) zero, if $\rho_1 \not\cong \rho_2$, or
- (ii) a scalar multiple of the identity, if $\rho_1 \cong \rho_2$ and $\mathbb{F} = \mathbb{C}$.

Remark 5.1: Why Schur’s Lemma matters for ML

Schur’s Lemma severely constrains the space of equivariant linear maps. This means equivariant neural network layers have *far fewer learnable parameters* than unconstrained layers — a form of built-in regularisation. The flip side is that designing architectures for non-trivial groups (e.g. $\text{SO}(3)$) requires decomposing representations into irreducibles, which involves Clebsch–Gordan coefficients and spherical harmonics.

6 The Invariance–Equivariance Tradeoff

Forcing invariance into a feature map comes at a cost: information is destroyed. Consider the example of *angles*: a representation of a 2D vector $v \in \mathbb{R}^2$ that is $\text{SO}(2)$ -invariant can only retain the *magnitude* $\|v\|$ (all directional information is lost).

In practice, a well-designed geometric network uses:

- **Equivariant intermediate layers**: features transform predictably and carry geometric information.
- **Invariant output head**: the final prediction is independent of the reference frame.

This is the backbone architecture of models such as **SE(3)-Transformers**, **DiffSBDD**, and **NequIP** (used in molecular dynamics).

Remark 6.1: Universal approximation under symmetry

A natural concern is whether the symmetry constraint limits what the network can learn. The reassuring answer is that under mild conditions, G -equivariant neural networks are *universal* on the class of G -equivariant continuous functions: any such function can be approximated to arbitrary accuracy by a sufficiently deep and wide equivariant network. So equivariance reduces the hypothesis space *without* losing expressivity on the relevant target class — exactly the bias–expressivity trade-off we want.

Exercises

1. Let $G = \{+1, -1\}$ act on $\mathbb{R}$ by multiplication ($g \cdot x = gx$). Which of the following functions $f : \mathbb{R} \rightarrow \mathbb{R}$ are invariant, equivariant (with the same action on the codomain), or neither?
 - (a) $f(x) = x^2$
 - (b) $f(x) = |x|$
 - (c) $f(x) = x^3$
 - (d) $f(x) = e^x$
2. A neural network layer for sets is defined as $f(X) = \sigma(W \cdot \text{mean}(X) + b)$, where $X = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ and $\text{mean}(X) = \frac{1}{n} \sum_i x_i$. Is f invariant or equivariant under permutations of the elements of X ? Justify your answer.
3. A convolutional layer with a 3×3 filter is applied to a 28×28 image. Argue informally (without formal proof) why this layer is translation-equivariant but *not* rotation-equivariant.
4. Let $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ be a continuous function that is $\text{SO}(2)$ -invariant. Show that f depends only on the magnitude $\|x\|$, i.e. $f(x) = h(\|x\|)$ for some continuous $h : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}$. (*Hint: use the orbit picture from Remark ??.*)
5. (*Challenge*) Let $G = \mathbb{Z}/2\mathbb{Z} = \{0, 1\}$ act on $\mathbb{R}^2$ by $0 \cdot (x, y) = (x, y)$ and $1 \cdot (x, y) = (-x, -y)$. Find all linear maps $f : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ that are equivariant with respect to this action. (*Hint: write f as a matrix A and impose $A(-x, -y) = -Af(x, y)$, i.e. $A(-v) = -Av$. What does this imply about A ?*)